

Summary

First Document: Improving cognitive-state analysis from eye gaze with synthetic eye-movement data

Since there is small dataset, we may want to use synthetic data but it should improve model performance. Most previous synthetic eye-tracking methods generate only:

- fixation locations
- fixation durations

They **cannot generate realistic raw gaze signals**, including:

- micro eye movements
- saccades
- gaze velocity
- eye movement noise

But SP-EyeGAN can generate raw eye movement sequences, separated into two:

1. FixGAN generates:
 - fixational eye movements
 - microsaccades
 - natural fixation noise
 2. SacGAN generates:
 - saccades
 - realistic acceleration
 - realistic velocity
 - correct amplitudes
- The pipeline is:

```
Fixation locations
    ↓
FixGAN → synthetic fixation signal

SacGAN → synthetic saccades

    ↓
Merge together

Synthetic raw eye movement
```

Data augmentation is actually done by using FixGAN and SacGAN algorithms. By these algorithms, new human-like eye movements are created and they aren't modified from the original data but completely new data.

Only used Gaussian noise (contrastive learning) for not remembering the same numbers and *learning* a reading profile.

The main goal is to create synthetic eye movements that are statistically similar and behaviourally realistic from real human eye movements, NN can learn useful patterns.

Second Document: Data Augmentation for Deep Learning-based Gaze Estimation

In this article, data augmentation is done with ResNet18 and AlexNet .

Different from the SP-EyeGAN(first document), it transforms the images by using different styles. Styles which are used are below:

Geometric transformations

- Flipping & Rotation: Rotating the image with a random number of degrees within -30 and 30 to make it represent the images that could happen in the test set. Flipping is also done only horizontally for the same reason. You should apply horizontal flipping if task does not depend on lateralization (in our case, it is applicable).
- Time warping: Time warping simulates differences in the temporal dynamics of eye movements, such as faster or slower gaze trajectories
- Translation: Moving the face slightly. Electrode placement changes between sessions.
- Random cropping: The model learns not to depend on one small region. Cognitive events may occur at slightly different times. This method is so useful that reduced overfitting %14 to %26.
- Random Erasing: Prevent over-reliance on one feature. Real EEG results have noisy electrodes and missing channels so it improves robustness.
- Circular Shift: This is basically same as the translation with a higher portion shifted away. It was more effective in ResNet18. It is because of layer numbers (mentioned in the article). **Appearance transformations**
- Gaussian blur & Gaussian Noise Injection: Simulate sensor/environment imperfections. Readers may read under some bad conditions.

This study shows that small geometric transformations provide the best improvement because they simulate realistic variations (e.g. shouldn't rotate the head image 180 degrees just to create new variation).

The most effective augmentations:

- **Horizontal flipping** (with corrected gaze labels)
- **Translation**
- **Random cropping**
- **Rotation**

Best performance is achieved by Horizontal flipping + Translation

- This method works with images and not suitable with numeric data.

Third Document: Confusion Prediction from Eye-Tracking Data: Experiments with Machine Learning

Confusion Detection: Detecting whether a person is confused or not based on the EEG recordings.

They used 2 data augmentation technique:

- SMOTE
- ADASYN

Which feature to augment and why:

- fixation: has a biological meaning. In this article, they removed observation exceeding 600 milliseconds.
- X-Y gaze coordinates: they are strongly correlated. ($r = 0.6$). Treat them as one.
- Fixation Path: No two paths are exactly alike. If task depends on **reading comprehension**, augment it. If task depends on **biometric identification**, do not augment it. Synthetic eye movement removes individuality. Fixation Path => (Participant A: Image → Text → Graph → Text → Image; Participant B: Text → Image → Graph → Text)

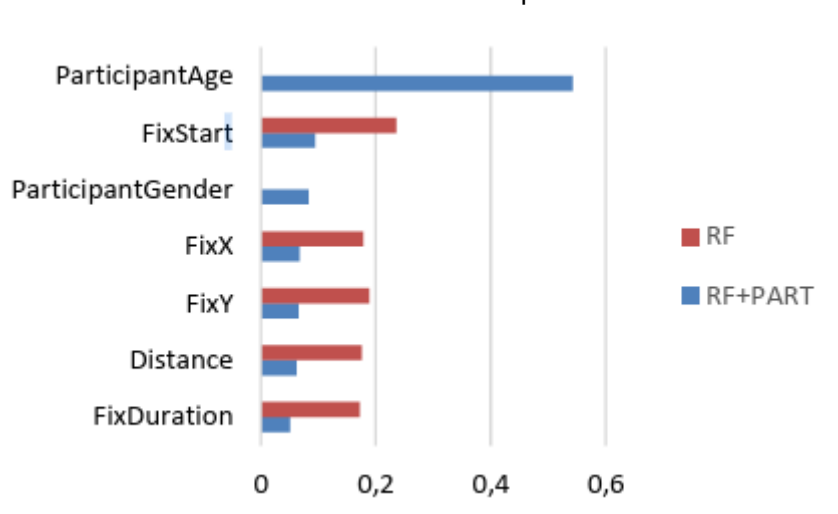
In overall, they didn't select a group of features, they used SMOTE and created a new class sample.

SMOTE provides higher performance. The model-training settings are **%67-%33** splitting and **10-fold cross-validation**.

Also tested with NN with layers from 3 to 10(best results on 3 layers). But RF performed better.

This comparison is done with the PART. (PART refers to including participant information like age, gender.)

Also when we checked what is the most important features in RF and RF+PART. The table is below:



A simpler model with good feature engineering can outperform deep learning.

Extra: Contrastive Learning

Contrastive learning is an AI training technique where models learn by comparing similar and dissimilar data points. It is self-supervised learning.

How to use contrastive learning in the eye-gazing dataset?

1. First define what your "sample" is

Each sample should represent one eye behaviour segment, for example:

```
[
  mean_fixation_duration,
```

```

fixation_count,
saccade_velocity,
saccade_amplitude,
scanpath_length,
x_mean,
y_mean,
pupil_size
]

```

(one row = one time window/session/trial) 2. Create augmentations (the most important part) Example:

Original:

```

fix_duration = 250 ms
saccade_velocity = 350 px/s
x_mean = 400

```

Augmented views:

```

View 1:
fix_duration = 255
saccade_velocity = 345
x_mean = 403

View 2:
fix_duration = 245
saccade_velocity = 360
x_mean = 398

```

Useful augmentations:

- **Gaussian noise** → simulate eye-tracker measurement noise
- **Feature masking** → randomly hide some features
- **Small scaling** → simulate natural variation in gaze behaviour
- **Time warping** (if using sequences)

3. Choose a contrastive learning framework Architecture:

```

Eye features
  |
  v
MLP encoder
  |
  v
Embedding vector
  |
  v
Contrastive loss

```

The model learns:

- Same eye behaviour → embeddings close

- Different eye behaviour → embeddings far apart

4. How do you define positive and negative pairs?

SCARF is designed for tabular data SimCLR is simpler approach Supervised Contrastive Learning is that if you have labels